



Shaik Baji

VULNERABILITY ASSESSMENT REPORT

**WEBSITE SECURITY AUDIT -
GINANDJUICE.SHOP**

20
26

Target: <https://ginandjuice.shop>

Date: May 2026

Tools: Nmap, OWASP ZAP,
Browser DevTools

Executive Summary

Date: May 2026

Prepared by: Shaik Baji

Target Website: <https://ginandjuice.shop>

Scan Type: Passive / Read-Only Assessment

Tools Used: Nmap, OWASP ZAP, Browser DevTools

A vulnerability assessment was conducted on ginandjuice.shop, a deliberately vulnerable web application by PortSwigger. The assessment identified 17 security alerts including High, Medium and Low risk vulnerabilities. This report documents all findings with risk classifications and remediation recommendations.

- High Risk: 3 findings
- Medium Risk: 8 findings
- Low Risk: 6 findings

Scope & Tools Used

SCOPE OF ASSESSMENT

- ✓ Target: <https://ginandjuice.shop>
- ✓ Scan Type: Passive / Read-Only
- ✓ Pages Tested: Public facing pages only
- ✓ Date: May 2026
- ✗ Login bypass - NOT performed
- ✗ Exploitation - NOT performed
- ✗ DoS attacks - NOT performed

TOOLS USED

- 🔧 Nmap 7.99 - Port & Service Detection
- 🔧 OWASP ZAP 2.17 - Vulnerability Scanning
- 🔧 Browser DevTools - Header Analysis

High Risk Findings

FINDING 1: SQL INJECTION

Risk Level: ● HIGH

URL: <https://ginandjuice.shop/catalog?category=%27>

Parameter: category

Evidence: HTTP 500 Internal Server Error

Description: Attacker can manipulate database queries to steal or delete data.

Fix: Use PreparedStatements, validate all inputs, never trust user input directly.

FINDING 2: CROSS SITE SCRIPTING (XSS)

Risk Level: ● HIGH

Description: Attacker can inject malicious scripts into web pages viewed by other users.

Fix: Encode all user output, implement CSP headers, validate and sanitize all inputs.

FINDING 3: VULNERABLE JS LIBRARY

Risk Level: ● HIGH

Description: Outdated JavaScript library with known vulnerabilities detected.

Fix: Update all JS libraries to latest versions, use dependency checking tools regularly.

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Quick Start Request Response Requester

Header Test Body Test

Header Text

Body Text

Requester

Header: HTTP/1.1 500 Internal Server Error

Date: Thu, 07 May 2020 11:31:34 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 3763

Connection: keep-alive

<!DOCTYPE html>

<html>

<!--LAB_HEAD_START-->

<head>

<link href=/resources/labheader/css/scanHeader.css rel=stylesheet>

SQL Injection

URL: https://ginandjuice.shop/catalog?category=%27

Risk: High

Confidence: Low

Parameter: category

Attack:

Evidence: HTTP/1.1 500 Internal Server Error

CWE ID: 89

WASC ID: 19

Source: Active (40018 - SQL Injection)

Input Vector: URL Query String

Description: SQL injection may be possible.

Other Info:

Solution: Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side.

Search Alerts Output Spider AJAX Spider Active Scan

Alerts (17)

Cross Site Scripting (Reflected) (3)

SQL Injection (2)

Vulnerable JS Library

Absence of Anti-CSRF Tokens (2)

Content Security Policy (CSP) Header Not Set (Systemic)

Cookie No HttpOnly Flag

Cookie Without Secure Flag

Cookie with SameSite Attribute None

Cookie without SameSite Attribute

Strict-Transport-Security Header Not Set

X-Content-Type-Options Header Missing

Cookie Poisoning (2)

Information Disclosure - Suspicious Comments (19)

Modern Web Application (Systemic)

Re-examine Cache-control Directives (Systemic)

Session Management Response Identified (68)

User Controllable HTML Element Attribute (Potential XSS) (4)

Cross Site Scripting (Reflected)

URL: https://ginandjuice.shop/login

Risk: High

Confidence: Medium

Parameter: username

Attack: 'alert(1)'

Evidence: 'alert(1)'

CWE ID: 79

WASC ID: 8

Source: Active (40012 - Cross Site Scripting (Reflected))

Input Vector: Form Query

Description: Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

Other Info:

Solution: Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoder, and the OWASP Java Encoder.

History Search Alerts Output Spider AJAX Spider Active Scan

Alerts (17)

Cross Site Scripting (Reflected) (3)

SQL Injection (2)

Vulnerable JS Library

Absence of Anti-CSRF Tokens (2)

Content Security Policy (CSP) Header Not Set (Systemic)

Cookie No HttpOnly Flag

Cookie Without Secure Flag

Cookie with SameSite Attribute None

Cookie without SameSite Attribute

Strict-Transport-Security Header Not Set

X-Content-Type-Options Header Missing

Cookie Poisoning (2)

Information Disclosure - Suspicious Comments (19)

Modern Web Application (Systemic)

Re-examine Cache-control Directives (Systemic)

Session Management Response Identified (68)

User Controllable HTML Element Attribute (Potential XSS) (4)

Vulnerable JS Library

URL: https://ginandjuice.shop/resources/js/angular_1-7-7.js

Risk: High

Confidence: Medium

Parameter:

Attack:

Evidence: AngularJS v1.7.7

CWE ID: 1395

WASC ID:

Source: Passive (10003 - Vulnerable JS Library (Powered by Retire.js))

Input Vector:

Description: The identified library appears to be vulnerable.

Other Info: The identified library angularjs, version 1.7.7 is vulnerable. CVE-2020-2336 CVE-2022-25869

Solution: Upgrade to the latest version of the affected library.

Medium Risk Findings

FINDING 4: ABSENCE OF ANTI-CSRF TOKENS

Risk Level: ● MEDIUM

Description: Forms do not have CSRF protection. Attacker can trick users into performing unwanted actions on the website.

Fix: Implement CSRF tokens on all forms, use SameSite cookie attribute.

FINDING 5: CONTENT SECURITY POLICY NOT SET

Risk Level: ● MEDIUM

Description: Missing CSP header allows attackers to inject malicious scripts easily.

Fix: Add Content-Security-Policy header to all responses, define trusted sources only.

FINDING 6: COOKIE NO HTTPONLY FLAG

Risk Level: ● MEDIUM

Description: Cookies can be accessed by JavaScript, allowing session theft.

Fix: Set HttpOnly flag on all sensitive cookies.

FINDING 7: STRICT TRANSPORT SECURITY MISSING

Risk Level: ● MEDIUM

Description: Site not enforcing HTTPS strictly, allowing downgrade attacks.

Fix: Add Strict-Transport-Security header with minimum age of 31536000 seconds.

History Search Alerts Output Spider AJAX Spider Active Scan

Alerts (17)

- Cross Site Scripting (Reflected) (3)
- SQL Injection (2)
- Vulnerable JS Library
- Absence of Anti-CSRF Tokens (2)
- Content Security Policy (CSP) Header Not Set (Systemic)
- Cookie No HttpOnly Flag
- Cookie Without Secure Flag
- Cookie with SameSite Attribute None
- Cookie without SameSite Attribute
- Strict-Transport-Security Header Not Set
- X-Content-Type-Options Header Missing
- Cookie Poisoning (2)
- Information Disclosure - Suspicious Comments (19)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)
- Session Management Response Identified (68)
- User Controllable HTML Element Attribute (Potential XSS) (4)

Absence of Anti-CSRF Tokens

URL: <https://ignandjuice.shop/catalog/product/productId=1>

Risk: Medium

Confidence: Low

Parameter:

Attack:

Evidence: `<form id="stockCheckForm" action="/catalog/product/stock" method="POST">`

CWE ID: 352

WASC ID: 9

Source: Passive (10202 - Absence of Anti-CSRF Tokens)

Input Vector:

Description:

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack

Other Info:

No known Anti-CSRF token [anticorsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, _csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "product"]

Solution:

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Reference:

History Search Alerts Output Spider AJAX Spider Active Scan

Alerts (17)

- Cross Site Scripting (Reflected) (3)
- SQL Injection (2)
- Vulnerable JS Library
- Absence of Anti-CSRF Tokens (2)
- Content Security Policy (CSP) Header Not Set (Systemic)
- Cookie No HttpOnly Flag
- Cookie Without Secure Flag
- Cookie with SameSite Attribute None
- Cookie without SameSite Attribute
- Strict-Transport-Security Header Not Set
- X-Content-Type-Options Header Missing
- Cookie Poisoning (2)
- Information Disclosure - Suspicious Comments (19)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)
- Session Management Response Identified (68)
- User Controllable HTML Element Attribute (Potential XSS) (4)

Content Security Policy (CSP) Header Not Set

URL: <https://ignandjuice.shop>

Risk: Medium

Confidence: High

Parameter:

Attack:

Evidence:

CWE ID: 693

WASC ID: 15

Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)

Alert Reference: 10038-1

Input Vector:

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page -- covered types are JavaScript,

Other Info:

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference:

History Search Alerts Output Spider AJAX Spider Active Scan

Alerts (17)

- Cross Site Scripting (Reflected) (3)
- SQL Injection (2)
- Vulnerable JS Library
- Absence of Anti-CSRF Tokens (2)
- Content Security Policy (CSP) Header Not Set (Systemic)
- Cookie No HttpOnly Flag
- Cookie Without Secure Flag
- Cookie with SameSite Attribute None
- Cookie without SameSite Attribute
- Strict-Transport-Security Header Not Set
- X-Content-Type-Options Header Missing
- Cookie Poisoning (2)
- Information Disclosure - Suspicious Comments (19)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)
- Session Management Response Identified (68)
- User Controllable HTML Element Attribute (Potential XSS) (4)

Cookie No HttpOnly Flag

URL: <https://ignandjuice.shop/resources/footer/js/canne.js>

Risk: Low

Confidence: Medium

Parameter: AWVSALB

Attack: Set Cookie: AWVSALB

Evidence: `1004`

CWE ID: 13

WASC ID: 13

Source: Passive (10010 - Cookie No HttpOnly Flag)

Input Vector:

Description:

A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.

Other Info:

Solution:

Ensure that the HttpOnly flag is set for all cookies.

Low Risk Findings

FINDING 8: INFORMATION DISCLOSURE

Risk Level: ● LOW

URL: ginandjuice.shop/resources/footer/js/scanme.js

Evidence: //TODO: Implementation here

Description: Developer left TODO comments in source code. Attackers can read these comments and gain information about unfinished features.

Fix: Remove all comments and TODO notes from production code before deployment.

FINDING 9: X-CONTENT-TYPE-OPTIONS MISSING

Risk Level: ● LOW

Description: Browser may interpret files differently than intended, enabling attacks.

Fix: Add X-Content-Type-Options: nosniff header to all server responses.

FINDING 10: REACT RUNNING IN DEV MODE

Risk Level: ● LOW

Description: Website is running React in development mode on production server.

This exposes debug information to attackers.

Fix: Build React app in production mode before deploying. Use `npm run build`.

History

Search

Alerts

Output

Spider

AJAX Spider

Active Scan

Alerts (17)

Cross Site Scripting (Reflected) (3)

SQL Injection (2)

Vulnerable JS Library

Absence of Anti-CSRF Tokens (2)

Content Security Policy (CSP) Header Not Set (Systemic)

Cookie No HttpOnly Flag

Cookie Without Secure Flag

Cookie with SameSite Attribute None

Cookie without SameSite Attribute

Strict-Transport-Security Header Not Set

X-Content-Type-Options Header Missing

Cookie Poisoning (2)

Information Disclosure - Suspicious Comments (19)

Modern Web Application (Systemic)

Re-examine Cache-control Directives (Systemic)

Session Management Response Identified (88)

User Controllable HTML Element Attribute (Potential XSS) (4)

Information Disclosure - Suspicious Comments

URL: <https://igmanduice.shop/resources/footer/js/canne.js>

Risk: Informational

Confidence: Medium

Parameter:

Attack:

Evidence: `!TODO: Implementation her`

CWE ID: 615

WASC ID: 13

Source: Passive (10027 - Information Disclosure - Suspicious Comments)

Input Vector:

Description:
The response appears to contain suspicious comments which may help an attacker.

Other Info:
The following pattern was used: `!TODO: Implementation here`, see evidence field for the suspicious comment in target

Solution:
Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

Reference:

History

Search

Alerts

Output

Spider

AJAX Spider

Active Scan

Alerts (17)

Cross Site Scripting (Reflected) (3)

SQL Injection (2)

Vulnerable JS Library

Absence of Anti-CSRF Tokens (2)

Content Security Policy (CSP) Header Not Set (Systemic)

Cookie No HttpOnly Flag

Cookie Without Secure Flag

Cookie with SameSite Attribute None

Cookie without SameSite Attribute

Strict-Transport-Security Header Not Set

X-Content-Type-Options Header Missing

Cookie Poisoning (2)

Information Disclosure - Suspicious Comments (19)

Modern Web Application (Systemic)

Re-examine Cache-control Directives (Systemic)

Session Management Response Identified (88)

User Controllable HTML Element Attribute (Potential XSS) (4)

Strict-Transport-Security Header Not Set

URL: <https://igmanduice.shop/resources/footer/js/canne.js>

Risk: Low

Confidence: High

Parameter:

Attack:

Evidence: `319`

CWE ID: 319

WASC ID: 15

Source: Passive (10035 - Strict-Transport-Security Header)

Alert Reference: 10035-1

Input Vector:

Description:
HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.

Other Info:

Solution:
Ensure that your web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

Remediation Summary

PRIORITY 1 - FIX IMMEDIATELY

- ✓ SQL Injection
 - Use PreparedStatements
 - Validate all user inputs
 - Never trust raw input
- ✓ Cross Site Scripting (XSS)
 - Encode all output
 - Implement CSP headers
 - Sanitize all inputs
- ✓ Vulnerable JS Library
 - Update to latest version
 - Run npm audit regularly

PRIORITY 2 - FIX SOON

- ✓ Add CSRF Tokens to all forms
- ✓ Set Content-Security-Policy header
- ✓ Set HttpOnly flag on cookies
- ✓ Enable Strict-Transport-Security
- ✓ Add X-Content-Type-Options header

PRIORITY 3 - FIX WHEN POSSIBLE

- ✓ Remove TODO comments from code
- ✓ Switch React to production mode
- ✓ Add X-Frame-Options header

Conclusion

ASSESSMENT SUMMARY

This vulnerability assessment was conducted on ginandjuice . shop using passive, read-only techniques. No exploitation or harmful activities were performed during this audit.

FINDINGS OVERVIEW

- High Risk: 3 vulnerabilities found
- Medium Risk: 8 vulnerabilities found
- Low Risk: 6 vulnerabilities found
- Total: 17 vulnerabilities identified

RECOMMENDATIONS

The website has several critical security issues that require immediate attention, particularly SQL Injection and Cross-Site Scripting (XSS). Fixing these vulnerabilities will significantly improve the security posture of the website.

DISCLAIMER

This report was prepared for educational purposes as part of a cybersecurity internship at Future Interns. All testing was conducted ethically and responsibly.

Prepared by: Shaik Baji

Date: May 2026

Organization: Future Interns